

MODULE 31 · DOMAIN 3 - CONTINUOUS IMPROVEMENT FOR EXISTING SOLUTIONS

Improving Performance

Measure first. The bottleneck is rarely where the team assumes it is.

DOMAIN 3 of 4 EXAM WEIGHT 25% MODULE 31 of 44

Optimising the wrong component costs money and changes nothing. **Find the constraint before you spend.**

Four resources, four signals

IF THE CONSTRAINT IS	LOOK AT	COMMON MISDIAGNOSIS
CPU	Utilisation, and run queue depth	Blamed for everything by default
Memory	Swap, and the agent's memory metric	Invisible without the CloudWatch agent
Disk	IOPS, throughput, queue length, burst balance	Read as a CPU problem, since waiting looks idle
Network	Packets per second, bandwidth, connection count	Ignored until a specific instance ceiling is hit

ON THE EXAM

A process waiting on disk shows low CPU. Low CPU with poor performance is a signal, not an all-clear.

Percentiles, not averages

- ◆ **An average hides the tail.** A service can average 90 ms while one request in a hundred takes four seconds.
- ◆ **An SLA is about the experience of individual users**, and every user in that unlucky one per cent had a bad one.
- ◆ **Track p50, p90, p99.** The gap between p50 and p99 tells you how consistent the system is.
- ◆ At scale, p99 is not rare. A million requests a day means ten thousand people in the tail.

ON THE EXAM

Averages are for capacity planning. Percentiles are for user experience, and an SLA is a statement about user experience.

Diagnosing a database

- ◆ **Database load, measured as average active sessions**, is the metric that matters. It shows sessions doing work at any moment.
- ◆ **Broken down by wait event**, it tells you whether sessions are waiting on CPU, I/O, locks or the network.
- ◆ It also breaks down by SQL statement, user and host, which usually identifies the offending query directly.
- ◆ **Currency note:** the Performance Insights console has been replaced by CloudWatch Database Insights. The concepts and the API are unchanged.

ON THE EXAM

Wait events are the whole point. Sessions waiting on locks and sessions waiting on I/O need completely different fixes.

Rightsizing, properly

- ◆ **Compute Optimizer analyses real utilisation** and recommends a type, with a projected performance risk attached.
- ◆ **Fourteen days of data minimum.** A recommendation from a quiet fortnight will undersize you for month-end.
- ◆ **Change one dimension at a time** and measure. Changing family and size together tells you nothing about which helped.
- ◆ Consider a different family rather than a smaller size. A memory-bound workload on a compute family is oversized by design.

ON THE EXAM

Rightsizing is not only shrinking. A workload constrained on the wrong dimension needs a different shape, not less of it.

Gotchas that look like application problems

- ◆ **T-family credit exhaustion.** Fast for twenty minutes, then throttled. The classic misdiagnosed performance fault.
- ◆ **gp3 provisions IOPS independently of size,** so growing the volume does nothing. That was gp2 thinking.
- ◆ **Instance network and EBS bandwidth ceilings are per instance type.** A small instance caps throughput regardless of the volume.
- ◆ Connection limits, not query speed, are often the database constraint. Module twenty one's RDS Proxy case.

ON THE EXAM

Each of these presents as "the application is slow" and none is fixed in the application. Check the platform before the code.

Reading the scenario

THE SCENARIO SAYS

Average latency is fine, users complain

Fast at first, then slow, on small instances

Database CPU low, performance poor

Volume enlarged, IOPS unchanged

Teams guessed instance sizes years ago

Read replicas added, no improvement

IT IS POINTING AT

Look at p99, not the mean

T-family credit exhaustion

Wait events: I/O or locks, not CPU

gp3 provisions IOPS separately

Compute Optimizer, 14 days of data

The bottleneck is writes, or connections

ON THE EXAM

Every row is a metric that rules something out. Find the number in the scenario that eliminates the obvious answer.

Meridian Freight's booking API reports average response time of 110 ms against a 500 ms target, and the dashboard has been green for months. Customer complaints about slowness have risen steadily, and the support team can reproduce it occasionally. Traffic is roughly 2 million requests a day.

What should the solutions architect do FIRST?

- ☐ A Examine p99 and p99.9 latency, since the average is hiding a tail
- ☐ B Scale the fleet up, since demand has clearly grown
- ☐ C Add an ElastiCache layer to reduce response times
- ☐ D Increase the response time target to 1 second

◆ PAUSE AND DECIDE

What should the solutions architect do FIRST?

- A** Look at the percentiles
A green average with rising complaints is the signature of a tail. At 2 million requests a day, 1% is 20,000 people.
- B** Scale up
Spends money before identifying the constraint. If the tail is caused by a lock or one slow dependency, more capacity changes nothing.
- C** Add caching
A plausible fix applied before diagnosis. It may help, and nobody yet knows which requests are slow or why.
- D** Loosen the target
Makes the number compliant by lowering the bar. The customers experiencing four-second responses are unaffected.

THE SIGNAL

Green dashboard plus rising complaints is a contradiction that resolves one way: the metric is not measuring what users feel.

A reporting application on t3.medium instances performs well each morning and becomes very slow by mid-afternoon, recovering overnight. CPU utilisation shows 100% during the slow period. Adding instances helps briefly, then the new instances slow down too. The workload is steady throughout the day.

What is the cause?

- ☐ A CPU credit exhaustion on the burstable instance family
- ☐ B A memory leak developing through the day
- ☐ C Database connection pool exhaustion
- ☐ D Network bandwidth limits on the instance type

◆ PAUSE AND DECIDE

What is the cause?

- A

 CPU credit exhaustion
Burstable instances spend credits above baseline and throttle when exhausted. Overnight idleness rebuilds them, which is why mornings are fine.
- B

 Memory leak
Would fit the daily pattern, and a fresh instance would start clean and stay fast. The scenario says new instances also slow down.
- C

 Connection pool
Would show as waiting rather than 100% CPU, and would not follow a daily recovery cycle.
- D

 Network limits
Would appear as throughput plateauing, not sustained full CPU. The workload is steady, so bandwidth is too.

THE SIGNAL

New instances slowing down too is the planted detail. It rules out anything that accumulates inside one instance.

An Aurora PostgreSQL cluster is reported as slow. CPU utilisation on the writer sits at 22%. Database load is high, and the breakdown shows most sessions waiting on lock events rather than CPU or I/O. One particular UPDATE statement dominates. The team has proposed a larger instance class.

What should the solutions architect recommend?

- ☐ A Scale to a larger instance class as proposed
- ☐ B Investigate and fix the contended UPDATE, since sessions are waiting on locks rather than resources
- ☐ C Add read replicas to spread the load
- ☐ D Increase the provisioned IOPS on the storage

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Larger instance

More CPU for a database using 22% of what it has. Sessions waiting on a lock wait exactly as long on a bigger machine.

C Read replicas

Move read traffic. The contention is on an UPDATE, which only ever runs on the writer.

B Fix the contended UPDATE

Lock waits are contention, not capacity. The breakdown has already named the statement, so the fix is in the query or the transaction.

D More IOPS

The wait events are locks, not I/O. Faster storage does not shorten a wait for another transaction to commit.

THE SIGNAL

Lock waits are the one bottleneck no amount of hardware fixes. That is why the wait-event breakdown matters more than CPU.

A workload needs 12,000 IOPS. It runs on a 500 GiB gp3 volume delivering 3,000 IOPS. An engineer has proposed enlarging the volume to 2 TiB, reasoning that a larger volume gives more IOPS.

The instance is an m6i.4xlarge.

What should the solutions architect advise?

- ☐ A gp3 provisions IOPS independently of size; provision 12,000 IOPS on the existing volume
- ☐ B Enlarge the volume to 2 TiB as proposed
- ☐ C Switch to io2 Block Express, since gp3 cannot reach 12,000 IOPS
- ☐ D Stripe four gp3 volumes in RAID 0

◆ PAUSE AND DECIDE

What should the solutions architect advise?

- A** Provision IOPS directly
gp3 decouples IOPS from capacity. You provision 12,000 on the existing 500 GiB volume and pay only for the extra IOPS.
- B** Enlarge the volume
gp2 thinking. Under gp2, IOPS scaled with size, so this instinct was correct until gp3. On gp3 it buys storage and no performance.
- C** io2 Block Express
gp3 reaches 80,000 IOPS since late 2025, from module sixteen. 12,000 is comfortably inside it, and io2 costs more.
- D** RAID 0 striping
The workaround from when a single volume could not deliver enough. Unnecessary complexity and a wider failure surface.

THE SIGNAL

This is gp2 muscle memory. Under gp2, 3 IOPS per GiB meant capacity was the lever; gp3 removed that coupling entirely.

A team added three read replicas to an RDS database after users reported slowness. Six weeks later performance is unchanged. Investigation shows the application sends all traffic to the writer endpoint, and 80% of database time is spent on write operations. The replicas sit at 3% utilisation.

What is the most useful conclusion?

- ☐ A The replicas were the wrong fix; the constraint is write throughput, which replicas do not address
- ☐ B Add more read replicas
- ☐ C Increase the replica instance size
- ☐ D Enable Multi-AZ on the writer

◆ PAUSE AND DECIDE

What is the most useful conclusion?

- A

 Wrong fix; writes are the constraint
Two facts confirm it: the application never uses the replicas, and 80% of database time is writes. Replicas address neither.
- B

 More replicas
More of a resource sitting at 3% utilisation. Three unused replicas do not become useful by becoming six.
- C

 Bigger replicas
Enlarges idle capacity. The replicas are not the constraint, and were never receiving traffic.
- D

 Multi-AZ
An availability improvement, not a performance one. The standby serves no read or write traffic.

THE SIGNAL

Six weeks of paying for three idle replicas is the cost of not measuring first. The scenario is this module's thesis.

A platform team runs 200 EC2 instances sized by developers three years ago. They want evidence-based rightsizing recommendations, and are concerned that a recommendation made now would be based on the quiet summer period rather than the December peak. The business will not accept degraded performance at peak.

What should the solutions architect recommend?

- A Use Compute Optimizer, but check the observation period and consider its longer look-back before acting
- B Apply Compute Optimizer's recommendations immediately across all 200 instances
- C Halve every instance size and monitor for problems
- D Wait until December and rightsize then

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A** Compute Optimizer, with the period checked
Uses real utilisation, and the team's concern is legitimate. A longer look-back covers the peak, and the projected performance risk guides how far to go.
- B** Apply immediately
Ignores a concern the team has correctly identified. Recommendations from a quiet fortnight will undersize the December peak.
- C** Halve everything
Guessing in the opposite direction from the original guess. It is the behaviour that produced the problem.
- D** Wait for December
Delays the saving by months, and rightsizing during peak trading is the worst possible window to change instance types.

THE SIGNAL

The team's worry is correct and the answer honours it. A recommendation is only as good as the period it observed.

What to carry out of this module

- ◆ **Measure before you spend.** Several options will make something faster; only one addresses the constraint.
- ◆ **Low CPU with poor performance is a signal.** A process waiting on disk or a lock consumes no CPU.
- ◆ **Percentiles, not averages.** An SLA is about individual users, and the tail is where they live.
- ◆ **gp3 decouples IOPS from size.** Enlarging the volume is gp2 muscle memory and buys nothing.

ON THE EXAM

Wait events are the most useful database signal. Lock contention is the one bottleneck that no amount of hardware fixes.